Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 4.1

по курсу «Методы оптимизации»

«Метод Хука и Дживса в 3D»

Студент группы ИУ9-82Б Шемякин В.А.

Преподаватель Посевин Д. П.

Москва 2026

1 Задание

Реализовать два варианта метода Хука и Дживса (с однократным движением по направлению и с многократным движением по направлению).

Траекторию движения вдоль минимизирующей последовательности отобразить в трехмерном пространстве, а значение целевой функции цветом. Значение целевой функции цветом показывается так: в начальной точке минимизирующей последовательности цвет траектории светлый, а при приближении к минимуму цвет траектории становится более темным.

2 Результаты

Исходный код программы представлен в листинге 1.

```
using Plots
using Colors

rosenbrock(x) = sum(100 * (x[i+1] - x[i]^2)^2 + (x[i] - 1)^2 for i in 1:length(x)-1)
schwefel(x) = 418.9829 * length(x) - sum(xi -> xi * sin(sqrt(abs(xi))), x)
rastrigin(x) = 10 * length(x) + sum(xi -> xi^2 - 10 * cos(2   * xi), x)

function explore(f, x, h)
    n = length(x)
    x_new = copy(x)
    probes = Tuple{Vector{Float64},Bool}[]
    for i in 1:n
        f_cur = f(x_new)
        p = copy(x_new); p[i] += h[i]
        if f(p) < f_cur
            push!(probes, (p, true))
            x_new[i] += h[i]
        else
            push!(probes, (p, false))
            p2 = copy(x_new); p2[i] -= h[i]
            if f(p2) < f_cur
                push!(probes, (p2, true))
                x_new[i] -= h[i]
            else
```

```
                push!(probes, (p2, false))
            end
        end
    end
    return x_new, probes
end

function search_single(f, x0, h0, eps; alpha=2.0, beta=0.5, max_iter=10000)
    n = length(x0)
    x_base = copy(x0)
    h = isa(h0, Number) ? fill(Float64(h0), n) : copy(h0)
    traj = [copy(x_base)]
    frames = []
    iter = 0

    while maximum(abs.(h)) > eps && iter < max_iter
        x_e, probes = explore(f, x_base, h)
        dest = f(x_e) < f(x_base) ? copy(x_e) : copy(x_base)
        push!(frames, (base=copy(x_base), probes=probes, tl=length(traj), dest=dest))

        if f(x_e) < f(x_base)
            x_pat = x_e + alpha * (x_e - x_base)
            x_base = x_e
            push!(traj, copy(x_base))

            x_pe, probes2 = explore(f, x_pat, h)
            dest2 = f(x_pe) < f(x_base) ? copy(x_pe) : copy(x_base)
            push!(frames, (base=copy(x_pat), probes=probes2, tl=length(traj), dest=dest2))

            if f(x_pe) < f(x_base)
                x_base = x_pe
                push!(traj, copy(x_base))
            end
        else
            h .*= beta
        end
        iter += 1
    end

    return x_base, f(x_base), traj, frames, iter
end

```

```
function search_multi(f, x0, h0, eps; alpha=2.0, beta=0.5, max_iter=10000)
    n = length(x0)
    x_base = copy(x0)
    h = isa(h0, Number) ? fill(Float64(h0), n) : copy(h0)
    traj = [copy(x_base)]
    iter = 0

    while maximum(abs.(h)) > eps && iter < max_iter
        x_e, _ = explore(f, x_base, h)

        if f(x_e) < f(x_base)
            d = x_e - x_base
            x_base = x_e
            push!(traj, copy(x_base))

            while iter < max_iter
                x_pat = x_base + alpha * d
                x_pe, _ = explore(f, x_pat, h)
                if f(x_pe) < f(x_base)
                    x_prev = x_base
                    x_base = x_pe
                    d = x_base - x_prev
                    push!(traj, copy(x_base))
                    iter += 1
                else
                    break
                end
            end
        else
            h .*= beta
        end
        iter += 1
    end

    return x_base, f(x_base), traj, iter
end

function seg_color_green(t)
    RGB(0.6 * (1 - t), 0.4 + 0.6 * (1 - t), 0.6 * (1 - t))
end
function seg_color_blue(t)
    RGB(0.6 * (1 - t), 0.6 * (1 - t), 0.5 + 0.5 * (1 - t))
end

```

```
function draw_3d_one(f, traj, title_str, filename; green=true)
    xs = [p[1] for p in traj]
    ys = [p[2] for p in traj]
    zs = [p[3] for p in traj]
    fv = [f(p) for p in traj]
    rng = max(maximum(fv) - minimum(fv), 1e-10)
    span_x = maximum(xs) - minimum(xs)
    span_y = maximum(ys) - minimum(ys)
    span_z = maximum(zs) - minimum(zs)
    pad = 0.2 * max(span_x, span_y, span_z, 0.5)
    xlim = (minimum(xs) - pad, maximum(xs) + pad)
    ylim = (minimum(ys) - pad, maximum(ys) + pad)
    zlim = (minimum(zs) - pad, maximum(zs) + pad)
    plt = plot(legend=false, xlabel=" x  ", ylabel=" x  ", zlabel=" x  ", title=title_str, xlims=xlim, ylims=ylim, zlims=zlim)
    for i in 1:length(traj)-1
        t = (fv[i] - minimum(fv)) / rng
        col = green ? seg_color_green(t) : seg_color_blue(t)
        plot!(plt, xs[i:i+1], ys[i:i+1], zs[i:i+1], seriestype=:path3d, color=col, lw=2)
    end
    savefig(plt, filename)
    display(plt)
end

function make_gif_3d_one(f, traj, title_str, filename; green=true, n_frames=80)
    L = length(traj)
    idx = L > 1 ? round.(Int, range(1, L, length=n_frames)) : fill(1, n_frames)
    xs = [p[1] for p in traj]
    ys = [p[2] for p in traj]
    zs = [p[3] for p in traj]
    fv = [f(p) for p in traj]
    rng = max(maximum(fv) - minimum(fv), 1e-10)
    span_x = maximum(xs) - minimum(xs)
    span_y = maximum(ys) - minimum(ys)
    span_z = maximum(zs) - minimum(zs)
    pad = 0.2 * max(span_x, span_y, span_z, 0.5)
    xlim = (minimum(xs) - pad, maximum(xs) + pad)
    ylim = (minimum(ys) - pad, maximum(ys) + pad)
    zlim = (minimum(zs) - pad, maximum(zs) + pad)
    anim = @animate for k in 1:n_frames
        i = idx[k]
```

```
        plt = plot(legend=false, title=title_str, xlabel=" x  ", ylabel=" x  ", zlabel=" x  ", xlims=xlim, ylims=ylim, zlims=zlim)
        if i >= 2
            for j in 1:i-1
                t = (fv[j] - minimum(fv)) / rng
                col = green ? seg_color_green(t) : seg_color_blue(t)
                plot!(plt, xs[j:j+1], ys[j:j+1], zs[j:j+1], seriestype=:path3d, color=col, lw=2)
            end
            ti = (fv[i] - minimum(fv)) / rng
            col = green ? seg_color_green(ti) : seg_color_blue(ti)
            scatter!(plt, [xs[i]], [ys[i]], [zs[i]], color=col, ms=5, marker=:circle)
        else
            col = green ? seg_color_green(0) : seg_color_blue(0)
            scatter!(plt, [xs[1]], [ys[1]], [zs[1]], color=col, ms=5, marker=:circle)
        end
    end
    gif(anim, filename, fps=12)
end

const show_plots = "--plot" in ARGS || "-p" in ARGS
const show_anim = "--anim" in ARGS || "-a" in ARGS
eps = 1e-6

funcs = [
    ("                    ", rosenbrock, [0.0, 0.0, 0.0], 0.4),
    ("                ", schwefel, [350.0, 400.0, 450.0], 0.5),
    ("                    ", rastrigin, [2.5, -1.5, 1.0], 0.3)
]

results = []

for (name, f, x0, h0) in funcs
    x1, fv1, t1, fr1, it1 = search_single(f, x0, h0, eps)
    x2, fv2, t2, it2 = search_multi(f, x0, h0, eps)

    println("\n$name:")
    println("                            :")
    println("    x* = ", round.(x1, digits=6))
    println("    f(x*) = ", round(fv1, digits=10))
```

```
    println("                    : ", it1)
    println("                                    : ", length(t1))
    println("                        :")
    println("    x* = ", round.(x2, digits=6))
    println("    f(x*) = ", round(fv2, digits=10))
    println("                    : ", it2)
    println("                                    : ", length(t2))

    push!(results, (name=name, f=f, t1=t1, t2=t2, x0=x0))
end

println("\n                                    :")
for r in results
    println("  $(r.name):                        = $(length(r.t1)), = $(length(r.t2))")
end

if show_plots
    draw_3d_one(rosenbrock, results[1].t1, "                    3D:                                    ", "lab4_rosenbrock_single.png", green=true)
    draw_3d_one(rosenbrock, results[1].t2, "                    3D:                                    ", "lab4_rosenbrock_multi.png", green=false)
    draw_3d_one(schwefel, results[2].t1, "                    3D:                                    ", "lab4_schwefel_single.png", green=true)
    draw_3d_one(schwefel, results[2].t2, "                    3D:                                    ", "lab4_schwefel_multi.png", green=false)
    draw_3d_one(rastrigin, results[3].t1, "                    3D:                                    ", "lab4_rastrigin_single.png", green=true)
    draw_3d_one(rastrigin, results[3].t2, "                    3D:                                    ", "lab4_rastrigin_multi.png", green=false)
    if show_anim
        make_gif_3d_one(rosenbrock, results[1].t1, "                    3D:                                    ", "lab4_rosenbrock_single.gif", green=true)
```

```
        make_gif_3d_one(rosenbrock, results[1].t2, "3D:", "lab4_rosenbrock_multi.gif", green=false)
        make_gif_3d_one(schwefel, results[2].t1, "3D:", "lab4_schwefel_single.gif", green=true)
        make_gif_3d_one(schwefel, results[2].t2, "3D:", "lab4_schwefel_multi.gif", green=false)
        make_gif_3d_one(rastrigin, results[3].t1, "3D:", "lab4_rastrigin_single.gif", green=true)
        make_gif_3d_one(rastrigin, results[3].t2, "3D:", "lab4_rastrigin_multi.gif", green=false)
    end
end
```

Результат представлен на рисунках 1– 3.

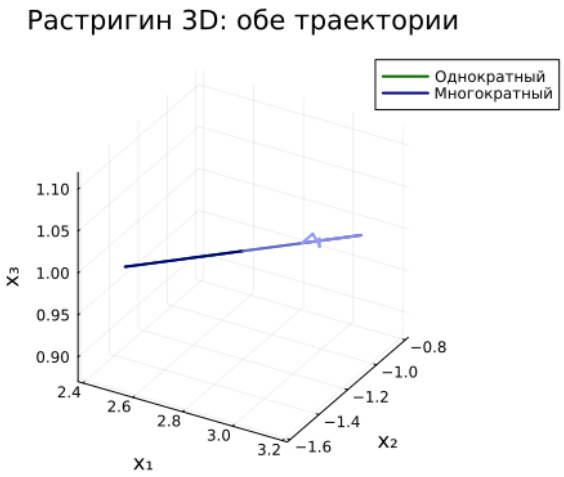

Рис. 1 — Результат

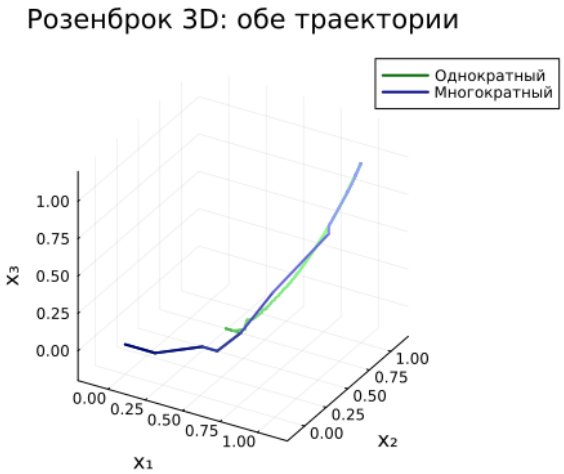

Рис. 2 — Результат

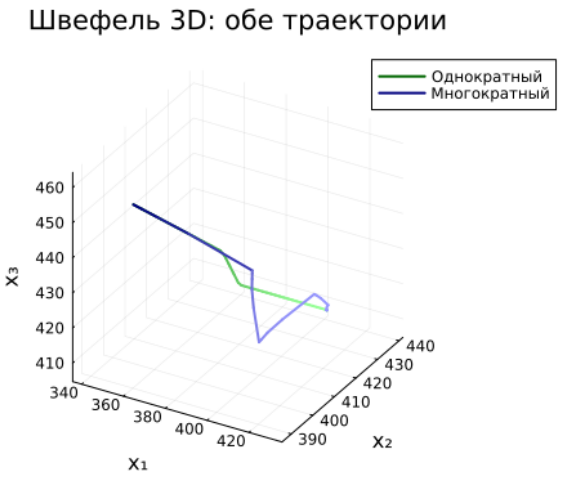

Рис. 3 — Результат